

Chapter 14. Object Oriented Programming & C++ Classes

Implementing the RGB Color Class

Topics: Classes • Static Components • Friend Functions • Operator Overloading • Color Theory • Bitwise Operations

1. Background

1.1 The RGB Color Model

Every color displayed on a modern screen is produced by combining three beams of colored light: **Red**, **Green**, and **Blue**. This is the **RGB additive color model**. Unlike mixing paints (which subtracts light), mixing light beams adds energy. Turn all three channels to full intensity and the result is white; turn them all off and you get black.

Each channel is represented by an integer in the range **[0, 255]** — eight bits, giving 256 levels of intensity per channel and a total palette of **$256 \times 256 \times 256 = 16,777,216$ colors**. A color is written in two equivalent notations:

```
Decimal notation : rgb(255, 165, 0)      ← orange
Hex notation     : #FFA500              ← same orange (FF=255, A5=165, 00=0)
```

Converting between the two is straightforward. Each pair of hex digits represents one channel:

```
Red   channel: FF (hex) = 15*16 + 15 = 255 (decimal)
Green channel: A5 (hex) = 10*16 + 5  = 165 (decimal)
Blue  channel: 00 (hex) = 0*16 + 0   = 0   (decimal)
```

1.2 Additive vs. Subtractive Color

Model	Used In	Primaries	Mix all →
RGB (additive)	Screens, monitors, LEDs	Red, Green, Blue	White
CMYK (subtractive)	Printing, ink, pigments	Cyan, Magenta, Yellow, K	Black (muddy)
RYB (traditional)	Painting, color wheels	Red, Yellow, Blue	Brown/Black

A common misconception: in RGB, **Yellow is not a primary color** — it is a mixture. Red + Green light at full intensity produces Yellow. This is why mixing colors in RGB follows different rules than mixing paints.

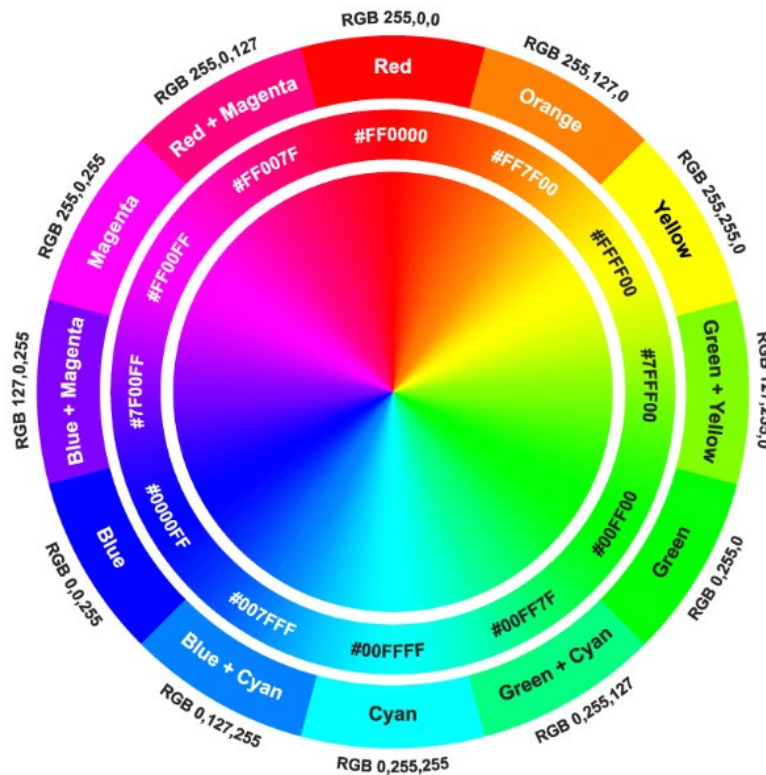


Figure 1. Color wheel

(Figure obtained from <https://martech.zone/complementary-color-palette-schemes/>)

Suggested Video.

COLOR THEORY BASICS: Use the Color Wheel & Color Harmonies to Choose Colors that Work Well Together, by Sarah Renae Clark. <https://www.youtube.com/watch?v=Yel6Wqn4I78>

1.3 YUV Encoding and Broadcast Color

Professional video systems — including streaming services such as **Netflix**, **Disney+**, and broadcast television — do not transmit raw RGB data. Instead, they use **YUV encoding** (also called YCbCr), which separates an image into:

- ▶ Y — Luminance (brightness). Human eyes are most sensitive to this component.
- ▶ U (Cb) — Blue-difference chrominance (color information, blue axis).
- ▶ V (Cr) — Red-difference chrominance (color information, red axis).

Because the human visual system is far more sensitive to brightness than to color detail, YUV allows broadcasters to transmit the Y channel at full resolution while *reducing* the U and V channels — a technique called **chroma subsampling**. This cuts bandwidth by up to 50× with minimal perceived quality loss.

The conversion from RGB to YUV follows a linear transform:

$$\begin{aligned} Y &= 0.299 R + 0.587 G + 0.114 B \\ U &= -0.147 R - 0.289 G + 0.436 B \\ V &= 0.615 R - 0.515 G - 0.100 B \end{aligned}$$

Notice the Y formula: Green contributes the most (58.7%) because the human eye has the highest sensitivity to green wavelengths. This same formula is used in our `toGrayscale()` method.

1.4 Impact on LCD and OLED Display Technology

LCD panels (Liquid Crystal Display) produce color by shining a white backlight through a matrix of liquid-crystal cells, each filtered with a Red, Green, or Blue color filter. The RGB triplet for each pixel controls how much light passes through each sub-pixel. One challenge with LCDs is that the backlight is always on, so black pixels are produced by blocking light rather than emitting none — limiting contrast ratios.

OLED panels (Organic Light-Emitting Diode) eliminate the backlight entirely. Each sub-pixel is an organic compound that *emits* light directly when current passes through it. A pixel with `rgb(0, 0, 0)` simply receives no current and emits no light, producing **true black** and virtually infinite contrast ratios. This is why OLED screens look dramatically richer than LCDs in dark scenes — the difference between `rgb(0,0,0)` and `rgb(1,1,1)` is the difference between absolute darkness and the faintest glow.

Both technologies rely on the RGB model at the hardware level. The YUV-encoded video stream from Netflix is decoded back to RGB before being sent to the display driver.

1.5 Clamping Color Values

RGB channels must stay in the range **[0, 255]**. Values outside this range produce undefined or wrapped results. **Clamping** constrains any value to stay within a legal range:

```
clamp(300, 0, 255) = 255    // too high → capped at max
clamp(-10, 0, 255) = 0      // too low → raised to min
clamp(128, 0, 255) = 128    // in range → unchanged
```

In C++, the standard library provides `std::clamp()` from `<algorithm>`. Our `RGBColor` class applies clamping in every setter and constructor so that no invalid channel value can ever be stored.

1.6 Mixing Colors — Linear Interpolation

The simplest way to blend two colors is a **linear average** of each channel independently:

```
mix( rgb(255, 0, 0),  rgb(0, 0, 255) )
  = rgb( (255+0)/2, (0+0)/2, (0+255)/2 )
  = rgb( 127, 0, 127 )    ← purple
```

A more powerful version is **weighted linear interpolation (lerp)**, which controls how much of each color dominates using a parameter $t \in [0.0, 1.0]$:

```
mixWeighted(colorA, colorB, t)
    channel = colorA * t + colorB * (1 - t)
```

t = 1.0 → fully colorA

t = 0.5 → equal blend

t = 0.0 → fully colorB

Linear interpolation is one of the most widely used operations in computer graphics, animation, and game development. It is the engine behind color gradients, smooth transitions, and palette generation.

2. Lab Requirements

2.1 Class Specification

Implement a C++ class named **RGBColor** with the following specification. Place the class definition and implementation in a header file `RGBColor.h`.

Private data members

- int red — channel value, always in [0, 255]
- int green — channel value, always in [0, 255]
- int blue — channel value, always in [0, 255]

Constructors

```
RGBColor() // default: black rgb(0,0,0)
RGBColor(int r, int g, int b) // clamp each channel
RGBColor(unsigned int hexColor) // construct from 0xRRGGBB
```

Getters and Setters

```
int getRed() const; void setRed (int r);
int getGreen() const; void setGreen(int g);
int getBlue() const; void setBlue (int b);
```

Member Functions

```
RGBColor complement() const;
RGBColor mix(const RGBColor& other) const;
RGBColor mixWeighted(const RGBColor& other,
                     double t) const;
RGBColor toGrayscale() const;
unsigned int toHex() const;
string toHexString() const;
string toString() const;
```

Static Factory Methods

```
static RGBColor Black();      static RGBColor White();
static RGBColor Red();       static RGBColor Green();
static RGBColor Blue();      static RGBColor Yellow();
static RGBColor Cyan();      static RGBColor Magenta();
static RGBColor Orange();
```

Operator Overloads

```
RGBColor operator+(const RGBColor& other) const; // blend (mix)
bool      operator==(const RGBColor& other) const;
bool      operator!=(const RGBColor& other) const;
friend ostream& operator<<(ostream& os, const RGBColor& c);
```

2.2 Implementation Notes

- ▶ [object Object]: implement a private helper clampChannel(int value) that returns max(0, min(255, value)). Call it in every constructor and setter.
- ▶ [object Object]: use ostream and iomanip (setw, setfill, hex, uppercase) to produce the #RRGGBB string. Do NOT use sprintf.
- ▶ [object Object]: use the ITU-R BT.601 luminance formula: $Y = 0.299R + 0.587G + 0.114B$.
- ▶ [object Object]: simple average of each channel: $(A + B) / 2$.
- ▶ [object Object]: channel = (int)(this * t + other * (1-t)). Clamp the result.
- ▶ [object Object]: delegate to mix().
- ▶ [object Object]: delegate to toString().

3. Required Test Cases

Your main.cpp must demonstrate each of the following. Label each test with a comment.

Test 1 — Constructors and toString()

```
RGBColor black;           // default constructor
RGBColor orange(255, 165, 0); // parameterized
RGBColor gold(0xFFD700);   // hex constructor
RGBColor homer = RGBColor::Yellow(); // static factory

cout << black << endl;    // rgb(0, 0, 0)      #000000
cout << orange << endl;   // rgb(255, 165, 0)   #FFA500
cout << gold << endl;     // rgb(255, 215, 0)   #FFD700
cout << homer << endl;    // rgb(255, 255, 0)   #FFFF00
```

Test 2 — Clamping

```
RGBColor clamped(300, -10, 128);
cout << clamped << endl;           // rgb(255, 0, 128)  #FF0080
// Channels 300 and -10 must be silently corrected
```

Test 3 — Complementary Colors

```
RGBColor bartsShirt(255, 165, 0);    // orange
RGBColor margesHair(70, 130, 180);   // steel blue

cout << bartsShirt.complement() << endl; // rgb(0, 90, 255)
cout << margesHair.complement() << endl; // rgb(185, 125, 75)

// Verify: color + complement = White
RGBColor sum = bartsShirt.mix(bartsShirt.complement());
cout << sum << endl; // should be close to rgb(127, 127, 127)
```

Test 4 — Mixing Colors (linear and weighted)

```
RGBColor red   = RGBColor::Red();
RGBColor blue  = RGBColor::Blue();
RGBColor green = RGBColor::Green();

cout << red.mix(blue) << endl; // rgb(127, 0, 127) purple
cout << red.mix(green) << endl; // rgb(127, 127, 0) olive

// Weighted: 75% red, 25% blue
cout << red.mixWeighted(blue, 0.75) << endl; // rgb(191, 0, 63)

// Simpsons cast color mixing
RGBColor lisasDress(255, 0, 200);
RGBColor mixed = bartsShirt + lisasDress; // operator+
cout << mixed << endl;
```

Test 5 — Grayscale Conversion

```
RGBColor margesHair(70, 130, 180);
RGBColor gray = margesHair.toGrayscale();
cout << gray << endl; // rgb(119, 119, 119) #777777

// Verify all three channels are equal in grayscale
cout << (gray.getRed() == gray.getGreen() &&
        gray.getGreen() == gray.getBlue()
        ? "Grayscale OK" : "ERROR") << endl;
```

Test 6 — Hex Conversion (stringstream / iomanip)

```
RGBColor springfield(255, 215, 0); // gold
cout << springfield.toHexString() << endl; // #FFD700
cout << hex << springfield.toHex() << endl; // ffd700

// Round-trip: construct from hex, print decimal
RGBColor roundtrip(0xFF8000);
cout << roundtrip << endl; // rgb(255, 128, 0) #FF8000
```

Test 7 — Operator Overloads

```
RGBColor c1 = RGBColor::Red();
RGBColor c2(255, 0, 0);
RGBColor c3 = RGBColor::Blue();

cout << (c1 == c2 ? "c1 == c2 : SAME"      : "DIFFERENT") << endl;
cout << (c1 != c3 ? "c1 != c3 : DIFFERENT" : "SAME")      << endl;

RGBColor blended = c1 + c3;    // operator+
cout << blended << endl;      // rgb(127, 0, 127)

cout << c1 << endl;           // operator<< via toString()
```

Test 8 — Setters and Getters

```
RGBColor marge;
marge.setRed(70);
marge.setGreen(130);
marge.setBlue(180);
cout << marge << endl;           // rgb(70, 130, 180)  #4682B4

// Test setter clamping
marge.setRed(999);
cout << marge.getRed() << endl; // 255   (clamped)
```

Reflection Questions

1. Why must RGB channel values be clamped? What happens on real hardware if a channel exceeds 255?
2. The mix() method uses simple averaging. Describe one scenario where this produces a visually incorrect result and suggest a better approach.
3. Explain the difference between the RGB and YUV color models. Why does Netflix use YUV instead of transmitting raw RGB data?
4. Why does an OLED screen display true black while an LCD cannot? How does this relate to the RGB value rgb(0, 0, 0)?
5. What does the setw() manipulator do, and why must it be applied before each channel in toHexString()? (Hint: sticky vs. non-sticky manipulators)

Tip:

Test your complementary colors by verifying that mixing a color with its complement produces a value close to rgb(127, 127, 127) — a neutral gray. This is the digital equivalent of the color-wheel property that complementary colors cancel each other out.